为什么需要 Skill Engineering



本页讲解

比如，今天花了几轮对话把日报做好，明天换个 Chat，同样的日期、去重和平均耗时又得重新解释。我们可以把今天用过的数据、确认的规则和核对过的结果留下来，再请 AI 把处理方法整理成 Skill。以后拿到同类数据，就有一套现成的方法可以继续用。交给同事时，他也能顺着这些资料看懂这份日报是怎么算出来、怎么验过的。

继续阅读与实践：点击打开在线深入阅读

从一次数据处理任务说起



本页讲解

这 7 行就是我们交给 AI 的原始明细。T004 出现了两次，同一张工单只能算一次；T006 是 10 月 11 日创建的，也不该进入 10 日的日报。处理后剩下 5 张工单，其中 3 张已完成、2 张待处理。完成的 3 张分别用了 60、45、90 分钟，加起来 195 分钟，除以 3，平均就是 65 分钟。我们希望 AI 把明细整理成这样的日报，并让人能核对它怎么算的。把这些规则保存成 Skill，下次拿到新数据，就有了可以沿用的方法。

继续阅读与实践：点击打开在线深入阅读

Chat、Skill、Skill Engineering



本页讲解

刚才那份日报，在一个 Chat 里做对，就完成了当次任务。把规则和处理脚本保存成 Skill，下一次便有了可调用的方法。不过，另一位同事拿到新数据，也能做对吗？还要实际试一试。我们讲的 Skill Engineering，就是在沉淀方法之后，继续检查换对话、换数据、遇到异常时会发生什么。这样才知道，保存的方法离开原来的聊天还能不能工作。

继续阅读与实践：点击打开在线深入阅读

一个数据处理 Skill 的两个部分



本页讲解

算出 65 分钟之前，还有不少判断要做。统计哪一天、字段齐不齐、重复工单能不能合并，都影响最后的结果。假如同一个 ID 的两行耗时不同，就要停下来弄清原因，不能悄悄选一个。图里的 Workflow 管这些推进和停止的决定，Data Processing 按确认的规则去重、筛选和计算。两部分接起来，结果验过了，才把日报交出去。

继续阅读与实践：点击打开在线深入阅读

AI 推理 vs 确定性执行



本页讲解

拿刚才的平均耗时来说，195 除以 3，适合交给固定脚本，下一次也按同一规则计算。AI 则可以帮助理解需求：客户说的日期是创建日期，还是完成日期？字段名称换了，含义是否相同？把这些问题说清楚之后，再交给工具执行。脚本也可能写错分母，所以我们还要拿已知结果的样例核对，不能因为用了代码就省掉验证。

继续阅读与实践：点击打开在线深入阅读

一个好的 Skill 长什么样



本页讲解

SKILL.md 说明这项工作怎么开始、什么时候完成；详细口径放 references，处理代码放 scripts。图上的三个脚本名表示职责，参考包把它们合在 pipeline.py 中。tests 常放检查脚本的程序测试，evals 常放让 AI 实际完成任务的场景和判定标准。这是组织文件的惯例，目录都按需增加。参考包将程序检查、输入样例和已确认答案放在 tests，AI 任务与核对项放在 evals/evals.json，直接引用同一份样例和答案。实际结果另存包外的新目录。答案依据已确认的业务规则，不能照抄当前输出。

继续阅读与实践：点击打开在线深入阅读

Qoder 实操：沉淀一个 Skill



本页讲解

先在普通 Chat 中做对日报，再把规则和实际使用的脚本整理成 Skill ZIP，安装后开新 Chat。第一次还没保存规则，需要把业务口径说明白；安装后只需说“用 normal.json 生成 2026-10-10 的工单日报，保存到新的结果文件夹”。每次都重讲算法，就无法看出 Skill 是否真的保存了方法。先用 / 选择技能检查执行，再不选择技能，用自然语言试它能否自动找到。

继续阅读与实践：点击打开在线深入阅读

如何验证并改进 Skill



本页讲解

工单跟练先完成结果与安装复用的基础验证。理解 Evals，可以看一批固定的真实任务：选 10 个端到端需求，覆盖登录、支付、权限、超时和异常处理，用同一套 Tests 判定成功。假设 Skill V1 完成 6 个、V2 完成 8 个，成功率就是 60% 和 80%，提升 20 个百分点；同时比较质量、耗时、Token 和失败原因。Task eval 可以复用 tests 的断言，增加的价值在于整批任务的表现比较。Trigger eval 看是否正确触发，Production eval 观察真实使用中的质量、成本与失败。图中数字只是示例，需重复运行并控制模型、工具、数据和运行条件，才能判断改善是否稳定。

继续阅读与实践：点击打开在线深入阅读

开始实践与问题反馈



本页讲解

你可以从手头一件重复工作开始，不必一次做出一个大系统。选定任务后，用自评看看还缺什么，再准备几份能判断对错的样例。在新对话里跑通了，就请另一位同事拿同版本再试，看看哪些说明还不够。遇到问题，把脱敏输入、预期与实际结果和校验记录一起交给张坚、李琦。这样大家能重现问题，也更容易把修正补进下一版用例。

继续阅读与实践：点击打开在线深入阅读